



OASIS Security Report

Security Misconfiguration Security Analysis

Date: 2026-06-11 09:47:54 — **Model:** mistral:latest

Vulnerability: Security Misconfiguration

Project: clean_contracts

Analysis root: ../clean

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for Security Misconfiguration.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical		0
High		0

Medium**2****Low****0**[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.632
/work/project/dataset/clean/simple_dao_fixed.sol	0.617

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.632

Source lines: 1-42

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
' Bank bank;', 'function airDrop() supportsToken hasNoBalance public{ // In
the fixed version supportsToken comes before hasNoBalance'
```

Explanation

The contract has an insecure function ordering, where the 'supportsToken' check comes after the 'hasNoBalance' check. This allows for potential reentrancy attacks.

Impact

Information disclosure or unauthorized access

Entry point

airDrop() function

Execution path

```
```
+- airDrop()
 |
 v
 └─ supportsToken
 │ └─ Bank.supportsToken()
 │ └─ reentrancy_attack
 └─ hasNoBalance
 │ └─ require(tokenBalance[msg.sender] == 0)
 │ └─ reentrancy_attack
+- transfer token to the attacker ```
```

##### Exploitation steps

- Call 'airDrop()' function
- During execution, call vulnerable 'Bank.supportsToken()' and then manipulate the contract state to trigger the reentrancy attack

**Conditions**

Attacker can call `airDrop()` multiple times in a reentrant manner

**Remediation**

'`hasNoBalance`' should come before '`supportsToken`' in the '`airDrop()`' function

**Secure example**

```
function airDrop() hasNoBalance supportsToken public{ //... }
```

## File 2: /work/project/dataset/clean/simple\_dao\_fixed.sol

Similarity score: 0.617

Source lines: 1-34

### ► Finding 2.1: Vulnerability found (Medium)

#### Vulnerable code

```
donate(address to) payable public {
 credit[to] += msg.value;
}
```

#### Explanation

Function donate is payable, allowing unchecked deposits into a public contract.

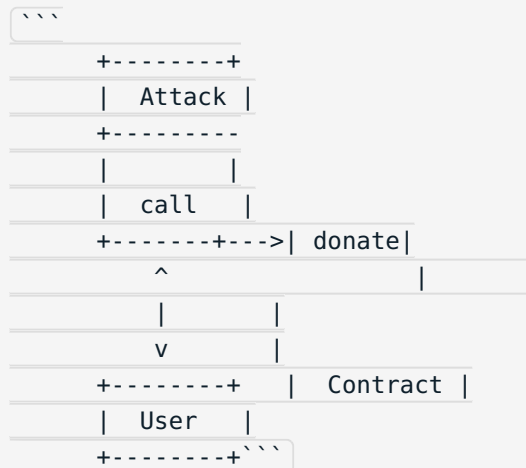
#### Impact

Unauthorized access to funds through the donate function.

#### Entry point

donate()

#### Execution path



#### Exploitation steps

- Call donate function with arbitrary value
- Deposit funds without authorization

#### Conditions

Attacker has knowledge of contract address and able to interact with it.

## Remediation

Remove payable attribute from the donate function or implement appropriate access control measures.

### Secure example

```
function donate(address to) public {
 require(msg.sender == owner, "Only owner can deposit.");
 credit[to] += msg.value;
}
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)